

LifeDrop

MERN Blood Donation & Transfusion Management System

Complete End-to-End System Architecture & Interview Preparation Manual

Generated for: Technical Deep-Dive & Engineering Interview Readiness

1. TECH STACK & SYSTEM HIGHLIGHTS

Core technologies powering the LifeDrop platform

LifeDrop is an enterprise-grade full-stack transfusion coordination network built to bridge voluntary blood donors, patients in critical care, hospitals, and licensed blood banks with real-time geospatial dispatch and strict state-machine controls.

Frontend Technology Stack:

- **React 19 & Vite 8:** Ultra-fast Single Page Application (SPA) architecture with ES module hot-reloading.
- **Tailwind CSS v3:** Utility-first modern design system supporting responsive dashboard grids.
- **React Router DOM v7:** Role-Based Access Control (RBAC) routing with ProtectedRoute and RoleRoute wrappers.
- **Axios Centralized Client:** Configured with request/response interceptors, automatic JWT silent refresh, and failed request replay.
- **Socket.io-Client:** Real-time WebSocket client receiving emergency broadcasts and live in-app notifications.
- **Recharts & Lucide React:** Interactive donor volume/inventory analytics visualizations and modern SVG iconography.

Backend Technology Stack:

- **Node.js & Express.js v4:** Modular RESTful API server implementing clean layered architecture (Routes -> Middlewares -> Controllers -> Services -> Models).
- **Mongoose v8 & MongoDB:** NoSQL persistence engine utilizing Geospatial 2dsphere indexes for coordinate-based radius matching.
- **JWT & Bcryptjs:** Dual-token authentication (15m Access Token + 7d Refresh Token) and 12-round salted password hashing.
- **Node-Cron Jobs:** Automated recurring tasks: daily inventory unit expiry checks, donor cooldown updates, and emergency radius auto-escalation.
- **Winston, Helmet & Express-Rate-Limit:** Structured logging, HTTP security header hardening, and anti-brute-force rate limiters.
- **ExcelJS & PDFKit:** Dynamic streaming export of clinical analytical reports.

2. COMPONENT & REPOSITORY STRUCTURE

Layered directory layout of client and server codebases

[Folder Tree Overview]

```
d:/Project/Blood Donation/
% % % client/                                [React 19 + Vite Frontend]
% % % % src/api/axios.js                     # Centralized Axios with Bearer injection & refresh replay
% % % % src/context/AuthContext.jsx          # Global session state & demo login personas
% % % % src/layouts/                         # PublicLayout, DashboardLayout, AdminLayout
% % % % src/pages/                           # Login, Register, Appointments, Inventory, etc.
% % % % src/routes/AppRoutes.jsx             # Route definitions with ProtectedRoute & RoleRoute
% % % % src/services/api.js                  # Clean modular API service abstractions
% % % server/                                [Node.js + Express Backend]
% % % % src/config/                          # Database (db.js), Socket (socket.js), Logger (logger.js)
% % % % src/models/                          # 19 Mongoose Models (User, Donor, Inventory, Request, etc.)
% % % % src/middlewares/                     # auth.js, validate.js, rateLimit.js, sanitize.js
% % % % src/routes/                          # 14 Route modules (auth, search, requests, emergency, etc.)
% % % % src/controllers/                     # Request handlers & HTTP responses
% % % % src/services/                       # Core business logic (matching, otp, email, reports)
% % % % src/jobs/                            # Node-cron background scheduled workers
% % % % src/server.js                        # Server entrypoint mounting WebSockets, cron & routes
```

3. END-TO-END ACTION FLOW: USER LOGIN

Here is the complete step-by-step lifecycle of a user authenticating into the LifeDrop system, from user input in the browser to database verification and state initialization:

- **1. UI Layer (Login.jsx):** User enters email and password into the login form. Upon submission, `handleSubmit(e)` invokes the `login()` action from `AuthContext`.
- **2. API Client (axios.js):** `AuthContext` dispatches a POST request to `/api/v1/auth/login` via the centralized Axios client.
- **3. Route & Middleware (auth.routes.js):** The request hits `authLimiter` (anti-brute-force) and `express-validator` to sanitize and validate email and password inputs before reaching the controller.
- **4. Controller (auth.controller.js -> login):** The login handler retrieves the user document using `User.findOne({ email }).select("+password +passwordHash +refreshToken")`.
- **5. Verification (User.js Model):** `user.comparePassword(enteredPassword)` is called, which runs `bcrypt.compare` against the 12-round salted hash. Verifies `user.isVerified` is true and `status` != "BLOCKED".
- **6. Token Generation (generateTokens):** Issues a 15-minute Access Token signed with `JWT_ACCESS_SECRET` and a 7-day Refresh Token signed with `JWT_REFRESH_SECRET`.
- **7. Audit & Persistence:** The refresh token is hashed/saved into the database (`user.save()`), and an immutable audit log entry (`LOGIN_SUCCESS`) is created.
- **8. Secure Response:** The server sets the Refresh Token in an `httpOnly`, `sameSite="strict"` cookie and responds with HTTP 200 containing the Access Token and User profile JSON.
- **9. UI Update & Redirect:** `AuthContext` persists the access token to `localStorage`, updates React state, displays a welcome toast via `react-hot-toast`, and redirects to `/dashboard` (or `/admin` for ADMIN role).

4. AUTHENTICATION & SECURITY ARCHITECTURE

Deep-dive into dual-token JWT, silent refresh, and RBAC guards

1. Dual-Token Architecture:

LifeDrop uses an industry-standard Dual-Token System that mitigates both Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF) vulnerabilities:

- **Access Token (Short-Lived: 15 Minutes):** Carries the identity payload: { id, email, role, name }. Attached to outgoing HTTP requests via the Authorization header (Bearer <token>). Because of its short lifespan, a compromised access token has limited exposure.
- **Refresh Token (Long-Lived: 7 Days):** Stored safely in an httpOnly cookie, preventing JavaScript-based extraction. Also persisted to the database to allow server-side revocation on logout.

2. Automatic Token Rotation & Request Replay (Silent Refresh):

When an access token expires during an active user session, Axios interceptors seamlessly renew it without disrupting the user:

[Axios Response Interceptor Workflow (client/src/api/axios.js)]

1. Request fails with HTTP 401 (TokenExpiredError).
2. Axios Response Interceptor catches 401, checks if request is already retried.
3. Original request is pushed into a pending failedQueue.
4. Axios dispatches a background POST /api/v1/auth/refresh-token with the refresh token.
5. Server verifies refresh token, updates database, and returns a new Access Token.
6. Interceptor updates Axios default headers with the new Bearer token.
7. processQueue() drains and replays all queued failed requests with the new token.
8. If refresh fails (e.g. 7 days elapsed), tokens are cleared and auth:unauthorized fires.

3. Role-Based Access Control (RBAC):

Access control is enforced at both the client routing layer and the server API middleware layer:

- **Server Middleware (middlewares/auth.js):** authenticate validates the JWT; authorize("ADMIN", "HOSPITAL") enforces required roles. Non-authorized roles receive HTTP 403 Forbidden.
- **Facility Safeguard (facilityAuth.middleware.js):** Hospitals and Blood Banks that register start with status="PENDING". They are strictly barred from inventory/request APIs until an Admin audits their license.
- **Client Guards (RoleRoute.jsx & ProtectedRoute.jsx):** Restricts React routes based on user authentication state and permitted roles, redirecting unauthorized users automatically.

5. MAIN REST API ENDPOINTS DIRECTORY

Primary endpoints, HTTP methods, and operational responsibilities

METHOD	ENDPOINT	DESCRIPTION
POST	/api/v1/auth/register	Self-serve registration for donors, recipients, hospitals, banks
POST	/api/v1/auth/verify-otp	Verifies 6-digit email OTP and activates user account
POST	/api/v1/auth/login	User & facility login (returns 15m access token + refresh cookie)
POST	/auth/admin/login	Strict dedicated admin login rejecting non-admin credentials
POST	/api/v1/auth/refresh-token	Rotates refresh token and issues fresh 15m access token
POST	/api/v1/auth/logout	Revokes active refresh token in database and clears cookies
POST	/api/v1/requests	Creates patient blood request (initial status: PENDING)
GET	/api/v1/requests/my	Fetches authenticated user/hospital blood requests
POST	/api/v1/requests/:id/confirm-received	Hospital confirms blood units received; marks FULFILLED
POST	/api/v1/emergency	Broadcasts emergency alert, queries nearby compatible donors
GET	/api/v1/emergency/nearby	Fetches active emergency alerts near donor coordinates
PUT	/api/v1/emergency/:id/respond	Donor accepts or declines an emergency blood broadcast
GET	/api/v1/appointments/slots	Returns available time slots and remaining capacity for blood bank
POST	/api/v1/appointments/book	Donor books voluntary donation appointment
PUT	/api/v1/appointments/:id/complete	Blood bank completes donation and increments inventory
GET	/api/v1/bloodbank/inventory	Returns stock counts across all 8 blood groups (A+, O-, etc.)
PUT	/api/v1/bloodbank/inventory/:group	Atomic inventory adjustment (ADD, REMOVE, SET)
POST	/api/v1/bloodbank/issue	Issues blood units against an approved blood request
GET	/api/v1/search/donors	Geospatial 2dsphere search for eligible donors (numbers masked)
GET	/api/v1/notifications	Fetches paginated user notifications (IN_APP)
GET	/api/v1/admin/stats	Returns system-wide metrics (donations, stock, emergencies)
PUT	/api/v1/admin/facilities/:id/verify	Admin approves pending hospital/blood bank license
GET	/api/v1/admin/audit-logs	Queries immutable system audit trail and action history
GET	/api/v1/admin/reports/export	Streams analytical reports in PDF or Excel (.xlsx) formats

6. GEOSPATIAL & REAL-TIME DISPATCH ENGINE

How emergency broadcasts and compatible donor matching work

Emergency Transfusion Matching Flow:

When an emergency occurs, seconds count. LifeDrop automates donor discovery through a 3-step pipeline:

- **1. Compatibility Matrix:** The matching service determines all compatible blood groups (e.g., O- is universal donor; B+ can receive from B+, B-, O+, O-).
- **2. Geospatial Radius Filter:** Queries MongoDB using the \$geoNear aggregation operator to find verified donors within a 15 km radius whose lastDonationDate was over 90 days ago (cooldown rule).
- **3. Multi-Channel Notification Dispatch:** Alerts are dispatched simultaneously via Socket.io WebSocket rooms, In-App Notification records, and email alerts via Nodemailer.
- **4. Automated Radius Escalation:** A Node-Cron worker checks open emergencies every 5 minutes. If required units are not fulfilled within 15 minutes, the search radius auto-escalates (15km -> 30km -> 50km).

7. TOP TECHNICAL INTERVIEW QUESTIONS & ANSWERS

High-impact questions interviewers ask about this project

Q1: How do you protect against race conditions during blood inventory updates?

Answer: We enforce atomic MongoDB updates using \$inc operators with conditional queries (e.g., unitsAvailable: { \$gte: requiredUnits }). If multiple requests attempt to issue blood units simultaneously, only the transaction finding sufficient stock succeeds; others fail safely without driving inventory below zero.

Q2: Why use both Access Tokens and Refresh Tokens instead of a single JWT?

Answer: A single long-lived JWT cannot be revoked if compromised without maintaining an expensive token blacklist. By using a short 15-minute Access Token alongside a long-lived Refresh Token in an httpOnly cookie, we limit the window of vulnerability. If suspicious activity occurs, the admin can revoke the refresh token in the database, locking the attacker out upon the next 15-minute rotation.

Q3: How are sensitive donor details protected during donor search?

Answer: Search results are scrubbed in the search service. Donor phone numbers and full addresses are masked (e.g., "98*****10"). Full contact details are only revealed if an official blood request between the donor and requester has reached the ACCEPTED or IN_PROGRESS status.

Q4: How does your application maintain real-time state synchronization?

Answer: We integrate Socket.io on both backend (server.js) and frontend (SocketContext.jsx). When an admin approves a facility or an emergency is created, the backend emits targeted events to specific user rooms (socket.to(`user\_\${id}`).emit(...)). The client listens to these events and updates local context state dynamically without requiring page refreshes.
